Enterprise API Gateway, Security Policy and Observability Control Plane

Architecture and End-to-End Engineering Guide

Broadcom_API_Management_Observability_Architecture

A reference implementation of API management and observability: Java 21 / Spring Boot control plane and gateway, Angular console, MySQL, Redis, Docker, Kubernetes, Helm, CI/CD, and a specification-driven, AI-assisted engineering workspace.

Scope and honesty: all data is synthetic and all credentials are dummy local-only values. This guide separates what was executed and verified (Java and Angular builds and tests, live gateway routing, static analysis, npm audit, end-to-end browser tests) from what was only authored and reviewed (Docker images, Kubernetes manifests, Helm chart, CI workflow, observability stack, Redis Lua scripts, MySQL migration test). See docs/validation/validation-report.md for the exact list.

Companion files: docs/architecture.md, docs/security.md, docs/adr/, docs/traceability-matrix.md, and the Deep Dive and Interview Guide PDF.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Contents

1. Business problem and project goals 3
2. Functional and non-functional requirements 3
3. Overall architecture 5
4. Control-plane and gateway responsibilities 6
5. Component and deployment diagrams 7
6. Database ER diagram 8
7. Request lifecycle from consumer to backend 9
8. Authentication and authorization flows 10
9. Policy configuration and propagation 11
10. Rate limiting and quota enforcement 12
11. Distributed tracing and observability 13
12. Failure detection and recovery 14
13. Docker and Kubernetes architecture 15
14. Helm deployment flow 15
15. CI/CD pipeline 16
16. Security threat model 17
17. Performance investigation workflow 18
18. API and GraphQL examples 19
19. Architecture Decision Records 20
20. Technical-leadership workflows 21
21. AI-assisted development lifecycle 22
22. Specification-driven development 22
23. Multi-agent orchestration 23
24. Context engineering and reusable AI skills 24
25. Code-review and vulnerability-remediation workflows 25
26. Architecture mind map 26
27. Troubleshooting scenarios 27
28. Limitations and future enhancements 28

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Business problem and project goals

An enterprise runs many backend services used by internal teams and external partners. Without a central platform every team re-implements authentication, throttling, routing rules, logging and change control - inconsistently - and nobody can answer basic questions such as "which APIs exist, who owns them, who may call them, what changed before the outage?". This project is a **reference implementation** of an API management and observability platform: a management **control plane**, a working **gateway (data plane)**, an Angular console, and the delivery assets to operate them.

It is inspired by the problem domain of enterprise API management and observability products and does not reproduce any vendor's proprietary implementation. All data is synthetic; every credential in the repository is a dummy local-only value.

Goals

- **Register and discover APIs** with ownership, versions, lifecycle and documentation.
- **Enforce policies on live traffic**: authentication, authorization, rate limits, timeouts, methods, size limits, IP restrictions, routing.
- **Change policies safely**: validate, four-eyes approve, version, apply, audit, revert.
- **Observe and investigate**: latency, errors, throughput, rate-limit and auth failures, traces, correlated config changes.
- **Operate it cloud-natively**: Docker, Kubernetes, Helm, CI/CD, secure defaults.
- **Show disciplined AI-assisted engineering**: specs, reusable context, review gates, honest evidence.

Explicit non-goals

- Reproducing a commercial product, a developer portal, billing, or multi-region active-active control planes.
- Claiming production readiness: `docs/leadership/production-readiness-assessment.md` lists the real gaps (no cluster run, no Docker validation, no live OIDC).

2. Functional and non-functional requirements

Area	Functional requirement	Where implemented
Registry	APIs with owner team, lifecycle (DRAFT/ACTIVE/INACTIVE/DEPRECATED), versions with unique base paths, multiple weighted targets, search/filter/pagination, optimistic locking	ApiService, ApiDefinitionRepository, V1 schema
Policies	Auth type, required role, methods, rate limit + algorithm + burst, timeout, max body, IP allowlist, retry flag; versioned; dry-run validation; submit/approve/reject/revert	PolicyConfig, PolicyValidator, PolicyService
Consumers	Consumers, subscriptions, API keys (hashed, expiring, revocable), shown once	ConsumerService

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Functional requirement	Where implemented
Gateway	Route match, rewrite, auth, limits, size/method/IP checks, timeouts, retries (safe only), breaker, bulkhead, health checks, SSRF guard, correlation ids, metrics	gateway module (GatewayFilter, ProxyService, ...)
Audit	Append-only record of privileged actions with actor and correlation id, atomic with the change	AuditService, AuditEvent
Observability	Request volume, error rate, latency percentiles, status distribution, rate-limit/auth failures, traces, structured logs	TelemetryRecorder, Prometheus, Grafana, OTLP
Incidents	Deterministic rules with evidence, recent config changes, correlated events	IncidentRules, IncidentService, UI
APIs	REST (+ RFC 7807 errors, idempotency), small read-only GraphQL, OpenAPI	web/ controllers, schema.graphqls
Console	Login, registry, detail/versions/targets, policy editor, approvals, consumers/keys, dashboard, events, incidents, audit, role-aware nav	frontend/src/app

Quality attribute	Non-functional requirement	How it is addressed
Availability	Traffic must survive control-plane, Redis and telemetry outages	Fail-static snapshot (memory + disk), Redis fallback modes, bounded async telemetry
Security	Deny by default, no unapproved destinations, no secrets in code, object-level authorization	Spring Security, AccessGuard, 3-layer destination allowlist, hashed keys, constant-time compares
Performance	Hot path O(1) in routes; bounded memory; no blocking on telemetry	Immutable route table, per-key locks, virtual threads, bounded buffers
Consistency	Config changes bounded-delay, never half-applied	Atomic swap, validate-then-apply, after-commit invalidation
Operability	Probes, graceful shutdown, dashboards, alerts, runbooks	Actuator groups, preStop + drain, Grafana JSON, Prometheus rules

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Quality attribute	Non-functional requirement	How it is addressed
Maintainability	Clear module boundaries, tests for every control	Architecture rules, 136 automated tests (101 Java, 27 Angular, 8 e2e) + a 32-check smoke script, ADRs

3. Overall architecture

The platform separates a low-traffic, stateful, transactional **control plane** from a high-traffic, stateless, latency-critical **data plane**. Gateways never query the database: they pull an immutable configuration snapshot, validate it, swap it atomically and keep serving the last-known-good copy if the control plane disappears.

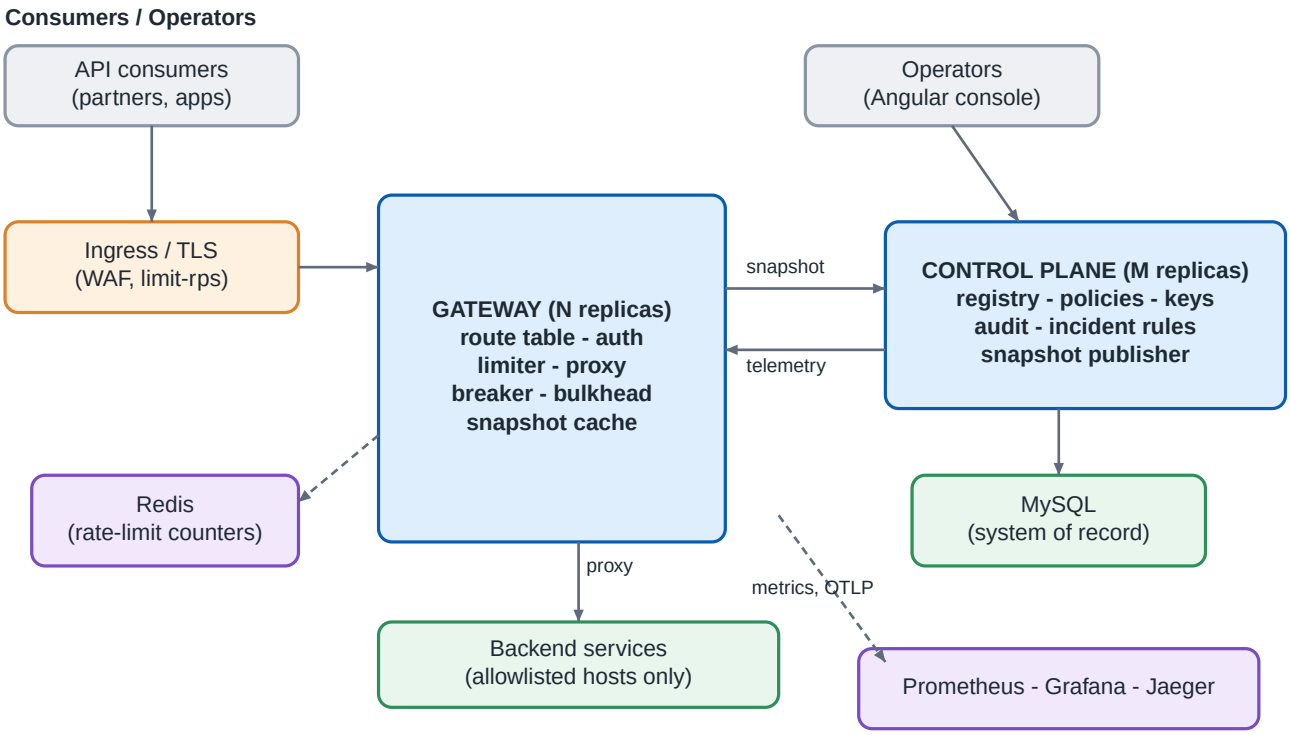


Figure 1. Component diagram: consumers enter through the ingress to the gateway; operators use the console and control plane; configuration flows down as snapshots, telemetry flows up as batches.

- **Why two deployables?** Scale, availability and release cadence differ (ADR-0001).
- **Why pull?** Idempotent, self-healing, outage-tolerant; costs a bounded propagation delay (ADR-0003).
- **Why a custom gateway?** Registry-driven policy pipeline, full control of limits/breaker/error contract, simple tests (ADR-0002); trade-offs recorded.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Control-plane and gateway responsibilities

Concern	Control plane	Gateway
State	MySQL (system of record), in-memory snapshot cache	Memory only: snapshot, breaker/health state, local limiter; optional disk copy of last-known-good
Primary job	Validate, approve, version, audit and publish configuration; consumer/key lifecycle; operator read models	Resolve route, authenticate, authorize, limit, validate, proxy, emit telemetry
Scaling driver	Operator load (low)	Consumer traffic (high); CPU-bound; HPA
If it fails	Changes and console stop; traffic continues; revocations delayed	Its share of traffic stops; other replicas continue
Security boundary	JWT (OIDC) for users, gateway token for /internal/	API key / JWT for consumers; never trusts inbound identity headers
Never does	Sit on the request path; call backends	Query MySQL; mutate live route tables; follow redirects

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. Component and deployment diagrams

Figure 1 (section 3) shows the logical components. The deployment view below shows the Kubernetes layout produced by `infrastructure/k8s` and the Helm chart: workloads are non-root with read-only root filesystems, a default-deny NetworkPolicy permits only the required flows, and backends are reachable only through the gateway.

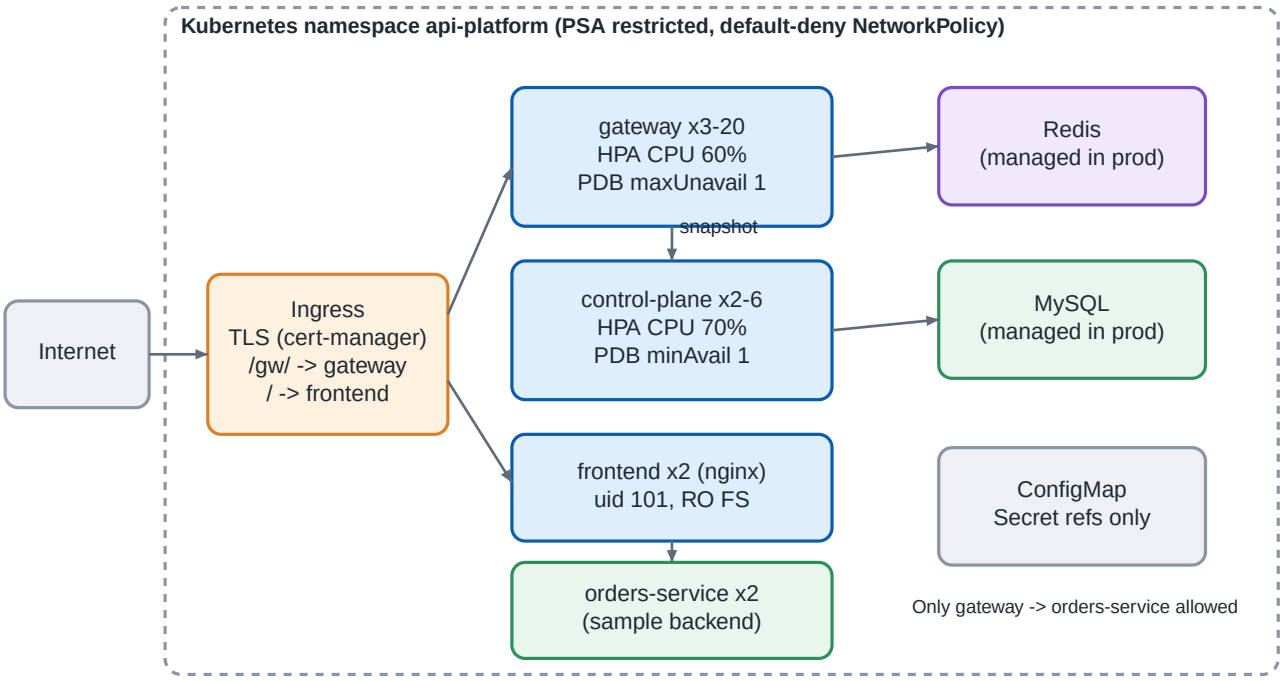


Figure 2. Deployment diagram (Kubernetes): ingress, gateway, control plane, frontend, sample backend, managed data stores in production, config by reference.

> **Caution:** These manifests and the Helm chart were authored and reviewed but **never applied to a cluster** in the build environment (no Docker, kubectl or helm available). CI contains the validation commands (`helm lint`, `kubeconform`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Database ER diagram

Schema source of truth: `control-plane/src/main/resources/db/migration/V1__core_schema.sql` plus `V2__audit_entity_time_index.sql` (a change driven by a query-plan review). Hibernate runs with `ddl-auto=validate`; Flyway owns the schema.

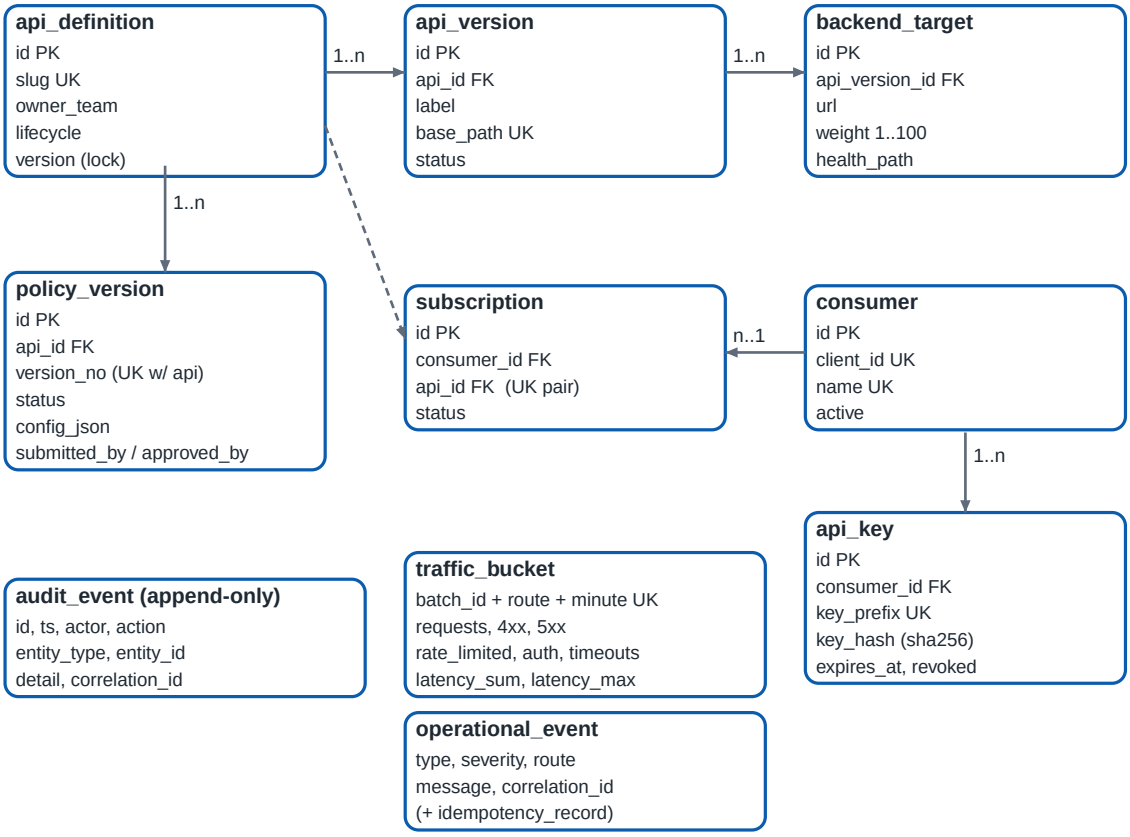


Figure 3. Entity-relationship overview (selected columns). Unique keys encode invariants: one gateway path per API version, one version number per policy history, one key prefix per key, idempotent telemetry per (batch, route, minute).

- **Optimistic locking:** `api_definition.version`; stale `PUT` returns `409 concurrent_modification`.
- **Audit:** append-only entity written in the same transaction (`Propagation.MANDATORY`); not purged by the application.
- **Retention:** telemetry 7 days, idempotency records 24 h (`RetentionJob`); audit follows a compliance-driven partition/archive plan.
- **Plan review:** `QueryPlanIT` asserts hot queries use their intended indexes (H2 plan shape only - confirm on MySQL with `EXPLAIN ANALYZE`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Request lifecycle from consumer to backend

The order of the gateway pipeline is deliberate: cheap, anonymous-safe checks run before expensive or stateful ones, and unauthenticated callers learn nothing beyond "not found / not allowed / unauthorized".

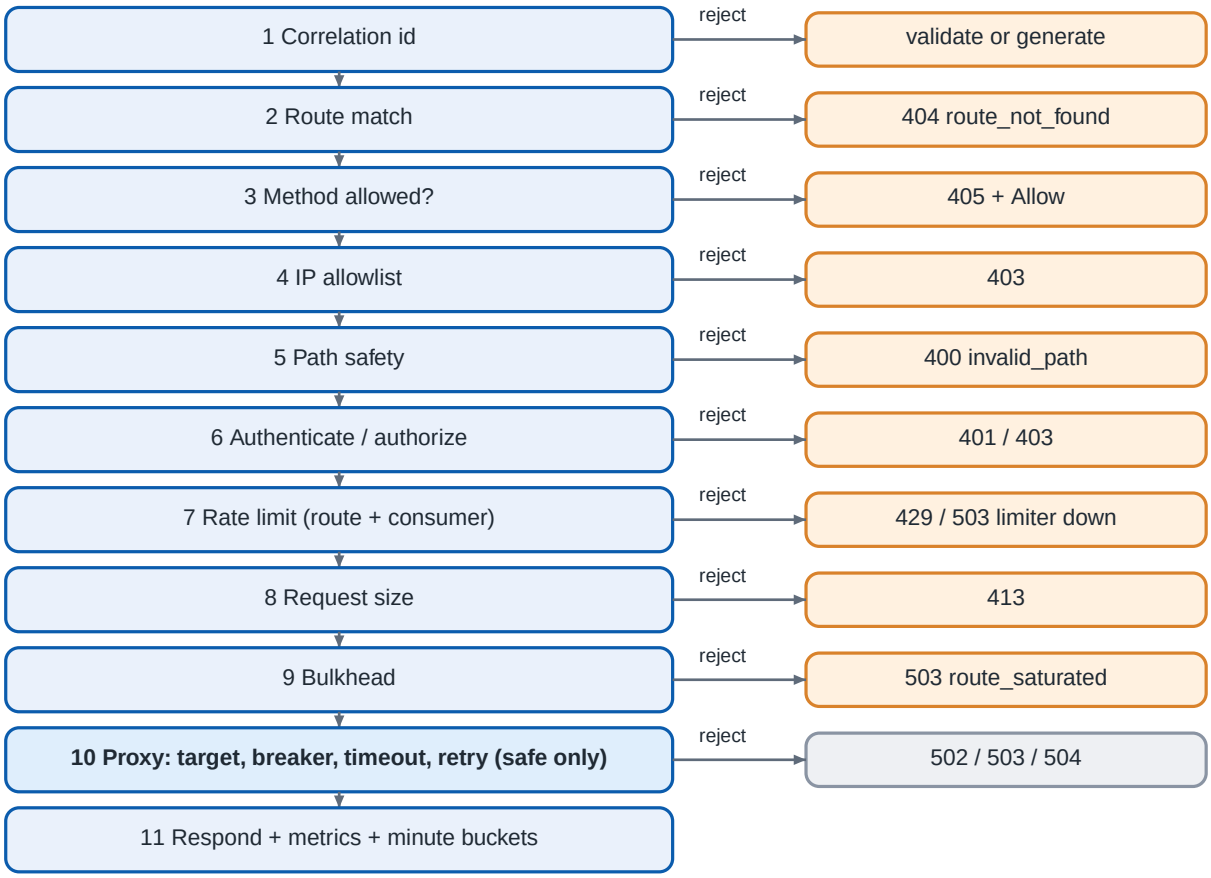


Figure 4. Gateway pipeline (GatewayFilter). Left: steps; right: the standardized rejection for each step. Step 10 applies timeout budget, safe retries and the circuit breaker.

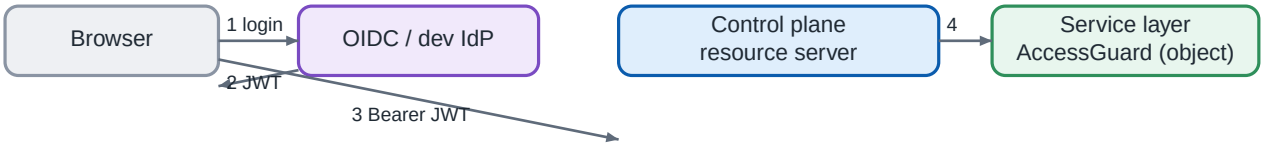
Proxy detail	Behaviour
Path rewriting	Base path stripped: /gw/orders/v1/items/7 -> backend /items/7 (query preserved); dot-segments rejected before forwarding
Headers	Hop-by-hop removed; inbound Authorization/X-API-Key/X-Consumer-Id/X-Forwarded-* discarded; gateway sets X-Consumer-Id, X-Correlation-Id, X-Forwarded-*, traceparent
Target choice	Smooth weighted round robin over healthy targets whose breaker allows traffic; none left -> 503 no_healthy_backend + Retry-After
Timeout and retry	policy.timeoutMs is an end-to-end budget; retries (<=3) only for GET/HEAD/OPTIONS on connect/502/503/504 with full-jitter backoff inside the budget

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Proxy detail	Behaviour
Limits	Request body bounded (declared and streamed); response capped at 10 MiB; per-route concurrency semaphore

8. Authentication and authorization flows

Operator (UI / REST / GraphQL)



JWT verified: signature, issuer, exp -> roles/team claims -> @PreAuthorize (role) -> AccessGuard (owner team) -> audit

Policy approval: submitter != approver (self_approval -> 403); gateway channel uses X-Gateway-Token (ROLE_GATEWAY only on /internal/**).

Consumer (data plane)

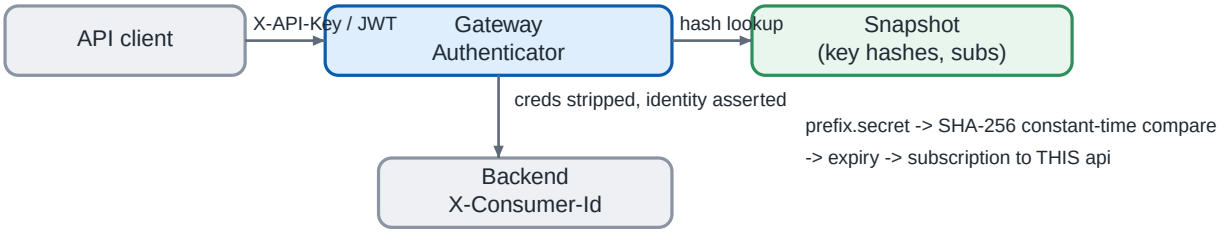


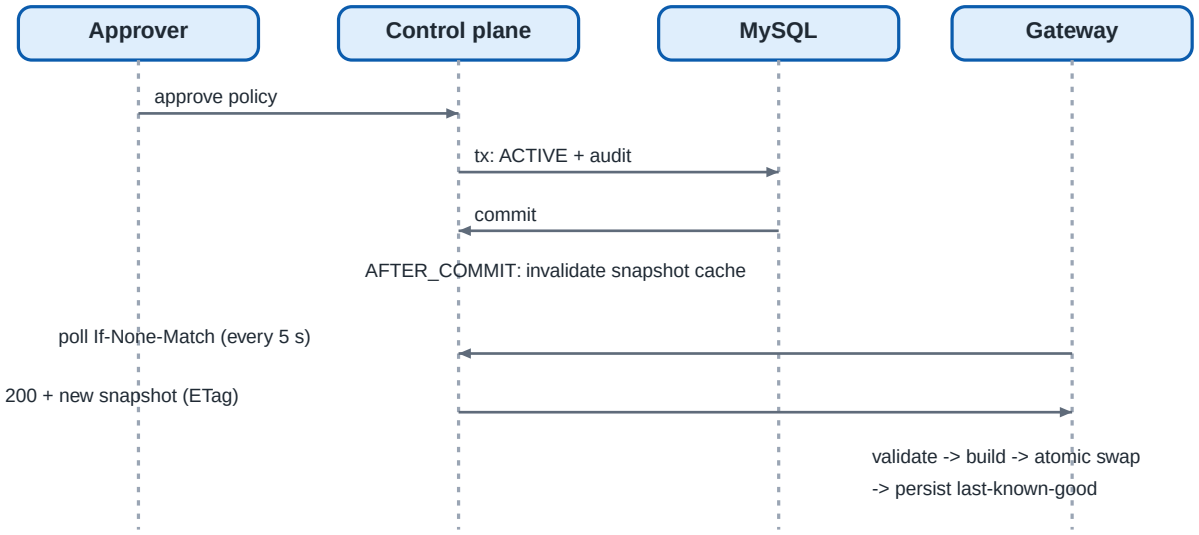
Figure 5. Operator flow (JWT, role + object-level checks) and consumer flow (API key hash lookup or JWT, subscription scoping, identity asserted to the backend).

- **Operators:** Spring Security resource server validates signature, issuer and expiry; roles/team claims drive `@PreAuthorize` and `AccessGuard` (BOLA defence). The dev identity provider issues the same claims an OIDC provider would; production switches to `issuer-uri` without code changes (ADR-0005).
- **Consumers:** API keys `prefix.secret` - 256-bit random secret, only SHA-256 stored, constant-time comparison (hash computed even for unknown prefixes), expiry, revocation, subscription to the specific API; JWT alternative verified on the gateway.
- **Internal channel:** `X-Gateway-Token`, constant-time, `ROLE_GATEWAY` only on `/internal/`**; mTLS/ workload identity in production.**
- **Four eyes:** a submitter cannot approve their own policy (403 `self_approval`).

> **Caution:** Demonstration security is not production security: dev HS256 secret, shared gateway token and HTTP on localhost. `docs/security.md` section 5 lists the production requirements.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Policy configuration and propagation



Propagation delay = poll interval (+ <= 30 s cache staleness on other CP replicas). Measured locally: ~7-8 s with a 2 s poll.

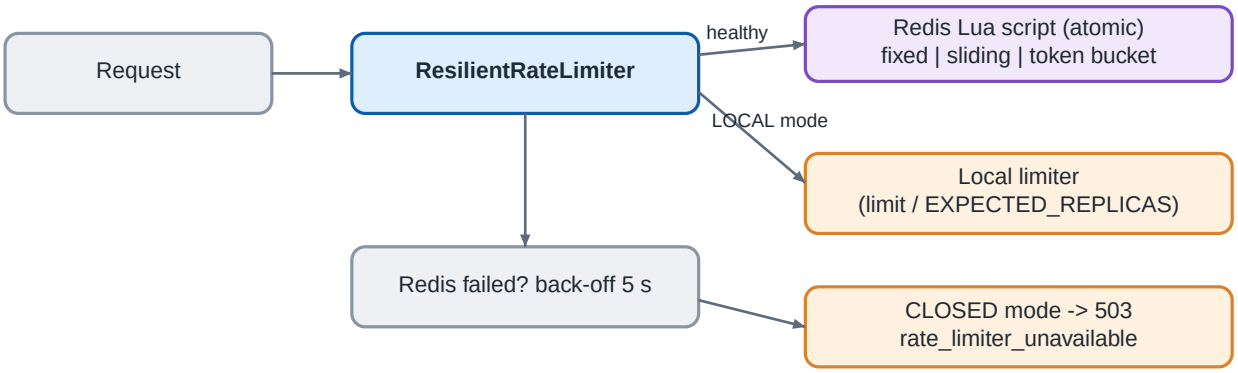
Control plane down: gateways keep the last-known-good config; revocations are delayed until recovery.

Figure 6. Policy approval and propagation. Delay = poll interval plus control-plane cache staleness; measured locally at about 7-8 s with a 2 s poll.

Question	Answer
Is propagation instantaneous?	**No.** Eventually consistent: bounded by the poll interval (default 5 s) plus up to 30 s cache staleness on other control-plane replicas. Different gateways may enforce different versions during the window.
What if the control plane is down?	Gateways keep serving the last-known-good snapshot (also persisted to disk); SNAPSHOT_STALE event after 3 failed polls; no changes - including revocations - take effect until recovery.
What if a snapshot is invalid?	The gateway validates every route/target; invalid routes are dropped with INVALID_CONFIG events; a snapshot with no valid routes is rejected whole and the previous config stays.
How is a bad change undone?	Revert creates a new version from an older one (append-only history), audited, propagates the same way.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10. Rate limiting and quota enforcement



Decision -> 200 + X-RateLimit-* | 429 + Retry-After. Redis health is NOT part of readiness (degrade, do not drop out of service).

Fixed window: boundary burst up to 2x. Sliding counter: weighted previous window. Token bucket: steady rate + bounded burst.

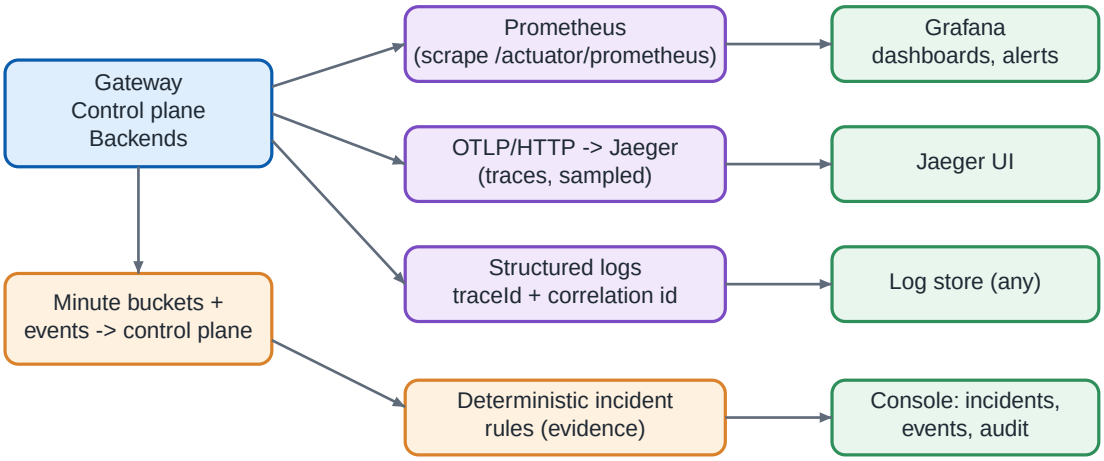
Figure 7. Resilient rate limiter: atomic Redis Lua decisions with a local fallback (LOCAL mode, divided limit) or fail-closed (CLOSED mode).

Algorithm	Behaviour	Weakness	Evidence
Fixed window	Count per 60 s window	Boundary burst up to 2x	fixedWindowAllowsBurstAtBoundary_knownWeakness
Sliding window counter	prev*(1-elapsed)+current	Approximation	slidingWindowRemovesTheBoundaryBurst
Token bucket	Refill limit/60s, capacity burst	Two parameters	tokenBucketAllowsBurstThenRefills...

- **Local vs distributed:** local limiting is exact per instance but admits up to N x limit across N gateways; Redis gives a global limit at the cost of a round trip and a new dependency (see `docs/rate-limiting.md`).
- **Concurrency:** 64 threads x 50 calls admit exactly the limit locally; Lua scripts make Redis decisions atomic (Redis test needs Docker and was skipped in the build environment).
- **Daily/monthly quotas** are specified (`specs/feature-quota-policy.md`) as the sample new-policy feature but **not implemented**.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11. Distributed tracing and observability



Labels: route = apiSlug/version (bounded) or "unmatched"; never path, consumer, IP, status code. Failures of any backend never block requests.

Figure 8. Telemetry paths: pull-based metrics, OTLP traces, structured logs with correlation ids, and per-minute aggregates feeding the deterministic incident rules.

- **Cardinality:** `route` label = registered `apiSlug/version` or `unmatched`; never raw paths, consumer ids, IPs or status codes. A test proves random 404 paths do not create series.
- **Sampling and cost:** head sampling configurable (0.5 compose, 0.05 Kubernetes); tail sampling in the OpenTelemetry Collector is the recommended production upgrade (keep all errors/slow traces).
- **Redaction:** no bodies, headers or credentials in logs/traces; correlation ids validated to prevent log injection.
- **Propagation:** `traceparent` injected on the proxied call from the gateway span; correlation id forwarded and echoed.
- **Verified:** metric names were checked against live services with `scripts/list-metrics.sh`; Grafana/ Prometheus/Jaeger themselves were not run (no Docker).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

12. Failure detection and recovery

Scenario	Detect	Contain	Recover / prevent
Backend down	TARGET_UNHEALTHY, CIRCUIT_OPEN, 5xx alert	Breaker fast-fails; bulkhead; other APIs unaffected	Half-open probe, health check; >=2 targets, readiness probes
Redis down	Redis failure counter + event	5 s back-off; LOCAL (divided limit) or CLOSED (503); Redis not in readiness	Auto recovery; managed Redis with failover
Slow DB query	Hikari pending, slow log	3 s connection timeout; traffic unaffected (no DB on data path)	EXPLAIN review, index migration, rollback
Gateway restart	Restart count, readiness flap	Graceful drain, maxUnavailable 0, PDB	On-disk snapshot cache; >=3 replicas
Invalid policy	422 at submit; INVALID_CONFIG at apply	Validation, four-eyes, last-known-good	Dry-run, contract tests, revert
Observability backend down	up==0, dropped counters	Async bounded exporters; never blocks requests	Redundant collectors; independent health checks
Retry storm	Retries >> requests, latency up	Safe methods only, <=3 attempts, jitter, end-to-end budget, breaker	Retry budgets (future), client backoff guidance

> **Note:** Full playbooks (detection, impact, containment, recovery, prevention) are in `docs/resilience-failure-scenarios.md`; ten troubleshooting exercises are in `docs/troubleshooting/`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13. Docker and Kubernetes architecture

- **Images:** multi-stage builds, Spring Boot layered jars, `eclipse-temurin:21-jre-alpine`, uid 10001, `MaxRAMPercentage=75`, `ExitOnOutOfMemoryError`, healthchecks; nginx-unprivileged for the SPA (CSP, same-origin API proxy; note `/api/` with trailing slash so it does not capture the SPA route `/apis` - a bug found by an end-to-end test).
- **Compose:** MySQL, Redis, two backends, control plane, gateway (+ second gateway under profile `ha`), frontend, Prometheus, Grafana, Jaeger; secrets from `.env` with `${VAR:?}`.
- **Kubernetes:** RollingUpdate (maxUnavailable 0), startup/readiness/liveness probes (gateway readiness requires a loaded snapshot; Redis excluded), preStop 10 s + 25 s drain < 40 s grace, requests/limits, HPA, PDB, topology spread, ServiceAccount per workload without API token, PSA `restricted`, default-deny NetworkPolicies, ingress TLS example.
- **Storage:** only MySQL is durable (managed service in production); Redis is intentionally ephemeral; gateway snapshot cache is an emptyDir.
- **Cloud mapping:** AWS primary (EKS, Aurora/RDS MySQL, ElastiCache, ALB+ACM, Secrets Manager via External Secrets, IRSA, ECR, managed Prometheus/Grafana) with Azure and GCP equivalents and local substitutes documented in `docs/kubernetes-and-helm.md`.

14. Helm deployment flow

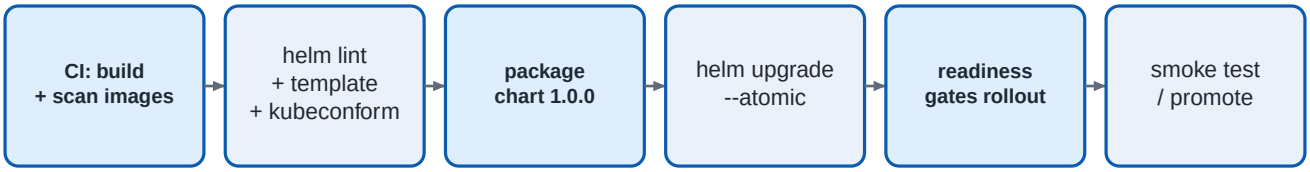


Figure 9. Helm delivery flow: images are built and scanned in CI, the chart is linted and rendered against schemas, then upgraded atomically so a failing readiness check rolls the release back.

- One generic template iterates over `components` (controlPlane, gateway, frontend, ordersService) producing ServiceAccount, Deployment, Service, PDB and HPA; separate templates create ConfigMap, Ingress and NetworkPolicies.
- `checksum/config` annotation rolls pods when configuration changes; secrets are referenced via `global.existingSecret`, never stored in values.
- `helm rollback api-platform <revision>` reverts; database migrations are expand/contract so N-1 and N coexist (move Flyway to a pre-install Job for strict change control).
- Status: chart reviewed only; `helm lint`/`template` not executed locally (no helm binary).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15. CI/CD pipeline

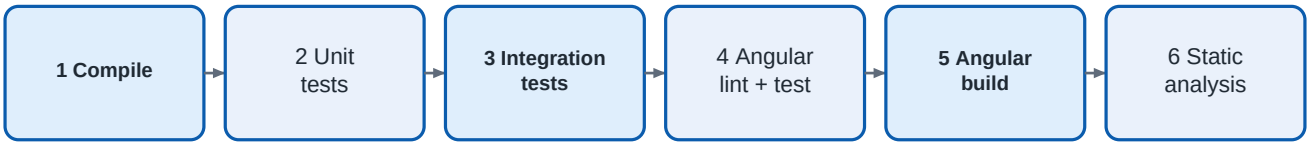


Figure 10. Pipeline stages 1-6 (GitHub Actions, `.github/workflows/ci.yml`).

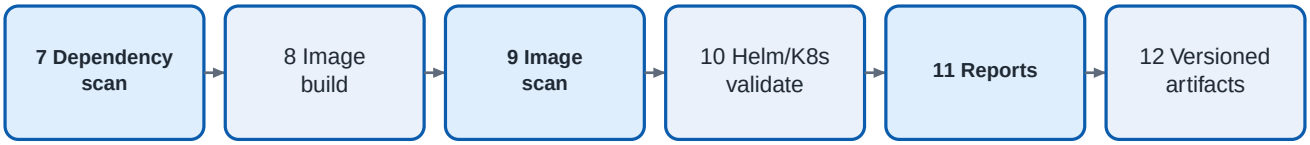


Figure 11. Pipeline stages 7-12: scanning, images, Kubernetes validation, reports and versioned artifacts.

Stage	Job	Executed locally?
1-3 Java compile, unit, integration	mvn compile / test / verify (Failsafe)	Yes (Docker-dependent ITs skipped)
4-5 Angular lint, tests, build	npm run lint / test:ci / build	Yes (Edge headless instead of Chrome)
6 Static analysis	SpotBugs profile; CodeQL	SpotBugs yes (0 high-confidence); CodeQL no
7 Dependency scanning	Trivy fs, CycloneDX SBOM	npm audit yes; Trivy/SBOM no
8-9 Images	buildx build, hadolint, Trivy image	No (no Docker)
10 Helm/K8s	helm lint, helm template | kubeconform, compose config	No (YAML syntax only)
11-12 Reports / artifacts	upload Surefire/JaCoCo/Karma coverage, versioned jars/dist/chart	Jars and reports yes

> **Note:** The pipeline was written and reviewed but has **never run on a CI runner**; expect small fixes on first execution. Least-privilege token permissions, pinned major action versions and no production credentials are by design; a deploy stage is intentionally left out (needs cluster credentials via OIDC federation).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16. Security threat model

Threat	Attack	Primary controls	Residual risk
Unauthorized access	Forged/expired token or key	JWT signature+issuer+exp, hashed keys, generic 401	Dev HS256 secret shared
BOLA (API1)	Owner of team A edits team B	AccessGuard in service layer; scoped sub-resources	Consumers not team-scoped (gap)
Credential leakage	Keys in logs/UI/repo	Shown once, hash at rest, masked URLs, no-store, .env ignored	Dummy dev defaults in local profile
Malicious route config	Target = metadata IP / internal host	3-layer allowlist + blocked ranges, audited rejections	DNS-rebinding window
SSRF via gateway	Proxy abuse	No redirects, connect-time re-resolve, NetworkPolicy egress	Private-IP names allowlisted by design
Denial of service	Floods, slow clients, big bodies, retry storms	Limits, bulkhead, body/response caps, timeouts, retry budget, breaker	Volumetric DDoS needs CDN/WAF
Info exposure	Verbose errors, label/PII leakage	RFC 7807 w/o internals, bounded labels, no payload logging	Logs hold correlation ids
Compromised dependency	Malicious/vulnerable library or image	BOM pinning, Trivy, SBOM, non-root minimal images	Maven scan only in CI
Misconfigured Kubernetes	Privileged pods, open network	PSA restricted, drop caps, read-only FS, default-deny	Env-var secrets visible to exec
Audit tampering	Edit history	Append-only entity, same-transaction write	DB admin can alter; ship to WORM

OWASP Top 10 (2021) and API Security Top 10 (2023) mappings are in `docs/security.md`; the security regression tests span `DestinationValidatorTest`, `PolicyValidatorTest`, `ControlPlaneApiIT`, `GatewayRoutingIT`, `PathSafetyTest` and the Angular interceptor/guard specs. **No external penetration test or secret-scanning tool was run.**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17. Performance investigation workflow

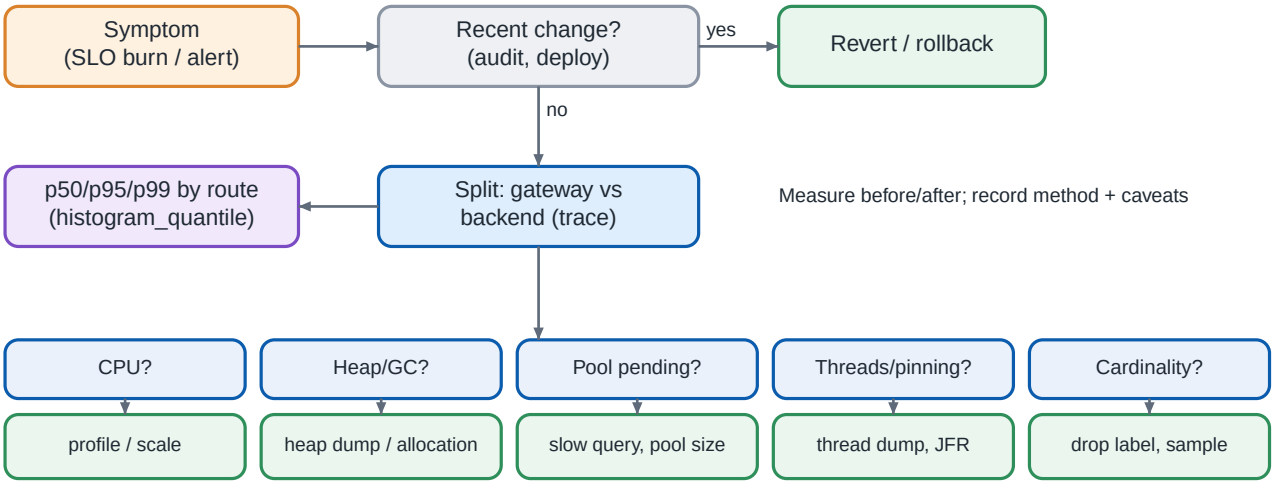


Figure 12. Investigation workflow: revert recent changes first, then split gateway vs backend time and walk the resource checks with evidence.

Measured results (laptop, shared CPU - relative only)

Scenario (32 workers)	req/s	p50	p95	p99
Backend directly	1,774	10.3 ms	50.2 ms	140.3 ms
Through gateway (3 runs)	919 / 1,204 / 1,353	25.2 / 16.3 / 12.9 ms	88.5 / 84.1 / 85.9 ms	149.6 / 177.8 / 194.8 ms
Gateway rejection: 401 (no key)	3,446	4.9 ms	36.0 ms	89.0 ms
Gateway rejection: 404	4,444	3.4 ms	22.4 ms	90.7 ms
Control plane list (16 workers, H2)	577	19.2 ms	75.9 ms	127.5 ms

> **Caution:** Client, gateway, control plane, two backends and the OS shared one 8-CPU laptop; closed-loop load; spread between runs was ~30%. k6 scripts exist but were **not executed**; no MySQL/Redis numbers were measured. Do not quote these as capacity. Raw output: `docs/validation/perf-*.txt`.

- **CPU saturation:** profile (JFR/async-profiler), scale gateways (HPA).
- **Memory/GC:** heap-after-GC trend, heap dump, allocation profile; every map/cache is bounded.
- **Pool exhaustion:** `hikaricp_connections_pending`, slow-query log, fail-fast 3 s acquire timeout.
- **Thread starvation:** virtual threads rarely starve; check pinning (`-Djdk.tracePinnedThreads`) and semaphore saturation.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Cardinality:** TSDB status, `count by (__name__)`, remove or aggregate labels.

18. API and GraphQL examples

Register, secure and call an API (all against the local profile with dummy credentials)

```
# 1. token (dev identity provider)
curl -s -X POST localhost:8080/api/v1/auth/token -H "Content-Type: application/json" \
     -d '{"username":"admin","password":"local-demo-password"}'

# 2. create an API (idempotent)
curl -s -X POST localhost:8080/api/v1/apis -H "Authorization: Bearer $T" -H "Idempotency-Key:
create-billing-0001" \
     -H "Content-Type: application/json" -d
'{"slug":"billing","name":"Billing","ownerTeam":"platform"}'
# 3. unsafe target is rejected (SSRF) -> 422 unsafe_destination
curl -s -X POST localhost:8080/api/v1/apis/$ID/versions/$VID/targets -H "Authorization: Bearer
$T" \-H "Content-Type: application/json" -d '{"url":"http://169.254.169.254","weight":1}'

# 4. call through the gateway
curl -i -H "X-API-Key: kdemo.local-demo-only-key-0123456789abcdef" localhost:8081/gw/orders/v1/
orders
```

Policy document (versioned, validated before activation)

```
{ "auth": "API_KEY", "allowedMethods": ["GET","POST"], "rateLimitPerMinute": 120, "burst": 20,
  "rateAlgorithm": "TOKEN_BUCKET", "timeoutMs": 3000, "maxRequestBytes": 65536,
  "ipAllowCidrs": null, "retryEnabled": true }
# POST /api/v1/policies/validate -> {"valid":true,"errors":[]}   (dry run, no side effects)
```

Standard error (RFC 7807 + code + correlation id)

```
{ "type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Policy validation
failed",
  "code":"invalid_policy","correlationId":"9c5f5ab1-9981-46f3-874a-6db5d91065a7",
  "errors":["auth=NONE is only allowed with read-only methods (GET, HEAD, OPTIONS)"] }
```

GraphQL: one round trip for the console view

```
query { apis(size: 5) { totalElements content { slug lifecycle versions { label basePath
targets { url weight } } } }
  healthSummary(windowMinutes: 15) { totalRequests errorRate routes { route requests
errors5xx avgLatencyMs } }~
```

REST vs GraphQL here: GraphQL is read-only and reuses the same service-layer authorization; state-changing operations stay REST where idempotency keys, optimistic locking and approvals have clear semantics (`docs/rest-graphql-protocols.md`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19. Architecture Decision Records

ADR	Decision	Main trade-off	Review trigger
0001	Separate control plane and data plane	Eventual consistency vs isolation/ availability	Need <1 s revocation; >50 gateways
0002	Custom gateway on Spring MVC + virtual threads	Own the edge cases (buffering, HTTP/1.1) vs simplicity and testability	Streaming/WebSocket/h2 needs
0003	Pull-based versioned snapshots + last-known-good	Bounded delay vs robustness	Fleet >100 or snapshot >5 MB
0004	MySQL + Flyway, no shared cache in front	Single-writer simplicity vs scale-out	Registry QPS / snapshot build > 100 ms
0005	OIDC/JWT for people, hashed API keys for consumers	Dev HS256 convenience vs production parity	Client-credentials needs; mTLS mandate
0006	Timeout budget, safe retries, consecutive-failure breaker, bulkhead	Simplicity vs partial-failure sensitivity	Missed partial failures
0007	Telemetry as batched idempotent HTTP, no broker	Possible loss vs zero extra infrastructure	Replay/multi-consumer analytics
0008	Destination allowlist at three layers	Operational friction vs SSRF defence in depth	Many namespaces; rebinding evidence
0009	Angular standalone, signals + RxJS	Token in sessionStorage vs simplicity	Shared state growth; Node upgrade
0010	Versioning and compatibility rules	Cost of multiple versions vs predictability	>2 live majors

Each ADR (in `docs/adr/`) records context, decision, alternatives, consequences and concrete review triggers; a blank template is in `docs/governance/adr-template.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20. Technical-leadership workflows

- **Feature incubation to delivery:** problem framing -> spec with acceptance criteria -> ADR for invariants -> vertical slices -> reviews -> release readiness -> post-release review. Roadmap, epics and cross-team dependencies: `docs/leadership/delivery-roadmap-and-epics.md`.
- **Review culture:** code-review checklist, architecture-review scenarios, AI-assisted PR disclosure (`docs/leadership/review-scenarios.md`).
- **Mentoring:** evidence-first coaching for recurring performance mistakes, security-change onboarding, review of AI-generated work (`docs/leadership/mentoring-examples.md`).
- **Communication:** weekly stakeholder update and release-risk summary templates that separate facts, estimates and unknowns.
- **Ownership:** the production-readiness assessment states plainly what is ready, partial or a gap - including the facts that Docker, Kubernetes, Helm and CI were never executed here.
- **Learning loop:** blameless post-incident review template with action types (prevent/detect/mitigate/process).

Phase	Theme	Exit criteria
0 Incubation	Framing, ADRs, walking skeleton	Scope and non-goals reviewed
1 MVP	Safe routing	Smoke + e2e green, threat model reviewed, one pilot API
2 Hardening	Operate it	Real OIDC, HA data stores, game day passed, SLOs agreed
3 Scale	Self-service, quotas	10+ APIs, onboarding < 1 day
4 Insight	Tail sampling, evidence-first analytics	Measured MTTR improvement

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

21. AI-assisted development lifecycle

AI assistants are used as **engineering collaborators inside a controlled loop**, not as one-off prompt generators: the repository carries reusable context, review modules and a verification discipline so any assistant (or human) can contribute safely.

1. **Context:** load `project-context.md` (short) + only the modules the task needs.
2. **Specify:** write or update the spec (acceptance criteria, security, failure behaviour, rollout).
3. **Implement in small tasks** with named files and tests.
4. **Verify with real commands** (tests, static analysis, smoke/e2e) and record the output.
5. **Review independently** against the checklists; treat generated code as untrusted.
6. **Human approval** for specs, designs, security-sensitive changes and releases.
7. **Capture learning:** update the module/checklist when a review finds a new failure mode.

> **Caution:** Honesty statement: in building this repository a **single assistant session** acted in every role sequentially; no parallel multi-agent execution took place. Two real defects were found by live runs rather than reviewed away (telemetry unique key; SPA route captured by `/api` proxy prefix) - each was fixed and is guarded by a test.

22. Specification-driven development

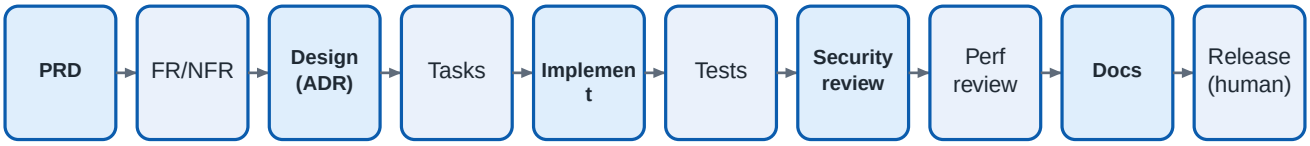
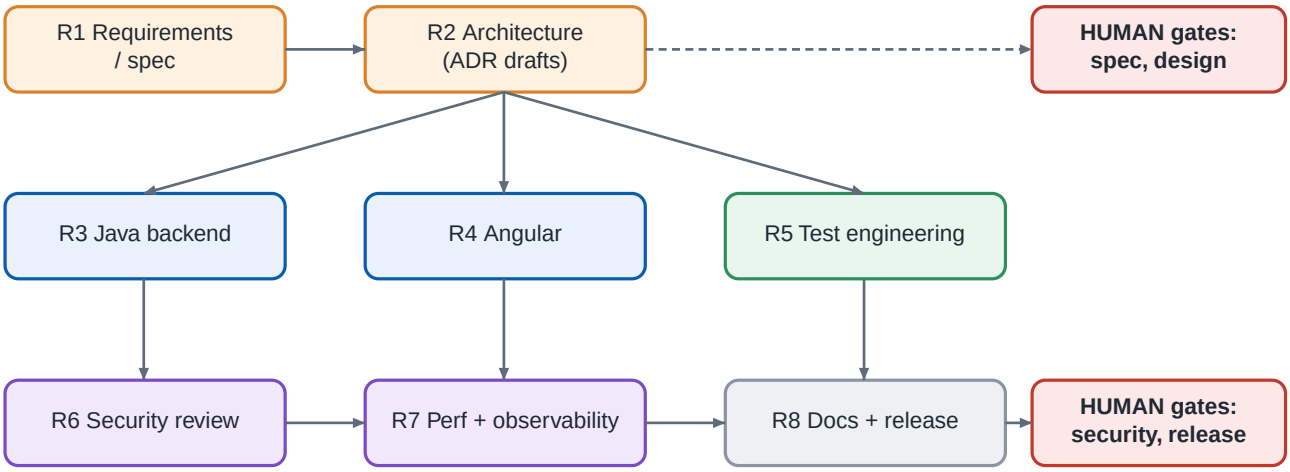


Figure 13. Ten-phase workflow (`ai-engineering/spec-driven-workflow.md`); humans approve the first three phases, security review and release.

- **Artifacts** live in `specs/<nnn>-<name>/` (prd, requirements, design, tasks, test-plan, security-review, perf-review, release).
- **Sample feature spec** for a new gateway policy type - daily request quota - with every required section: objective, stories, acceptance criteria, API contract, data model, security, failure behaviour, observability, performance expectations, test scenarios, rollout/rollback (`specs/feature-quota-policy.md`; intentionally not implemented).
- **Worked retrospective example:** `specs/001-ssrf-destination-guard/` shows all ten artifacts for a real feature, labelled retrospective.
- **Tool independence:** BMAD-style role phases or spec-kit style `specify/plan/tasks` map onto the same artifacts; names evolve, the artifacts are the contract.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

23. Multi-agent orchestration



Shared contracts (docs/api-contracts.md, ADRs, architecture-rules.md) are frozen inputs; file ownership per role; independent review for risky changes.

No multi-agent run was executed for this repository - this is the designed protocol with sample tasks/handoffs.

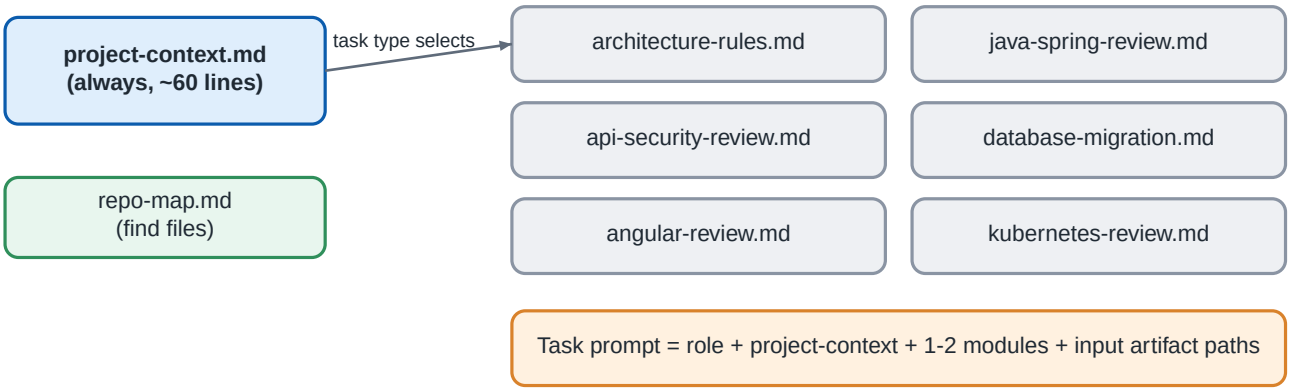
Figure 14. Designed multi-agent protocol: eight roles, dependency-aware sequencing, frozen shared contracts, human gates for specs/design and for security/release.

Role	May change	Must not change	Handoff criteria
R3 Java backend	control-plane/**, gateway/** (assigned files)	migrations merged earlier, infra, frontend, contracts	mvn verify output attached; no invariant violated
R4 Angular	frontend/	backend, contracts	lint/unit/build outputs
R5 Test engineering	src/test/**, e2e, performance**	production code	every acceptance criterion mapped; tests fail on mutation
R6 Security review	review docs only	production code	independent of author; blockers resolved or human-accepted
R7 Perf/ observability	observability/**, measured docs**	production code	numbers with method and caveats

- Agents never overwrite shared architectural decisions (ADRs, `architecture-rules.md`) and never approve their own high-risk work; conflicts escalate to the architecture role and then a human.
- Every handoff lists commands run and real results; unverifiable claims are marked UNVERIFIED (`ai-engineering/multi-agent/handoffs/_template.md`).
- **Sample tasks** T-101..T-103 and **sample handoff** H-101 are illustrative and say so; no multi-agent run was executed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24. Context engineering and reusable AI skills



Hand off summaries + paths (not transcripts); measure tokens from the tool usage report before claiming savings.

Figure 15. Modular context: one short always-loaded file plus task-selected modules and a repository map; hand off summaries, not transcripts.

- **Reusable modules (12):** project-context, architecture-rules, java-spring-review, angular-review, api-security-review, performance-review, test-generation, database-migration-review, kubernetes-review, incident-troubleshooting, feature-spec-template, release-readiness.
- **Tool adapters:** `CLAUDE.md` (Claude Code), `AGENTS.md` (generic), `.github/copilot-instructions.md`.
- **Techniques:** repository map instead of tree scans, smallest-range file reads, tests as documentation, ADRs as the memory of decisions, handoff notes, task splitting, detecting missing context (invented APIs, unverifiable claims).
- **Before/after files:** `ai-engineering/context-examples/` (monolithic vs modular).
- **Token measurement method** (no invented savings): same tasks, same model, fresh sessions, read input/output tokens from the tool usage report, compare medians and pass rate, record dates and model versions. **No token measurements were taken for this repository.**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

25. Code-review and vulnerability-remediation workflows

Generated code is untrusted until reviewed and validated: assistants optimise for plausible patterns, can invent APIs, miss invariants, and write tests that mirror the implementation. `ai-engineering/review-examples/examples.md` holds twelve flawed-to-fixed examples (SQL injection, BOLA, unbounded retries, non-atomic limiter without timeouts, SSRF, secrets in logs, check-then-act race, boundary violation, unbounded read, unsafe config swap, mirror tests, over-engineering), each tied to a real regression test.

Remediation item	Evidence (real unless marked)
Confirmed finding	`npm audit`: 14 high-severity in **dev tooling only** (braces via karma/typescript-eslint, GHSA-vfj7-8cjw-p6xm); production dependencies: **0**; no non-breaking fix (only `--force` downgrading karma) -> documented acceptance with owner and expiry
Dependency upgrade	java-jwt 4.4.0 -> 4.6.2 in the gateway: tests added first (nbf in future, alg:none), baseline and post-upgrade runs both green; transitive set unchanged; **performance not re-measured**; no vulnerability in 4.4.0 was claimed
Refactoring	TargetSelector rewrite during review (honestly noted as refactor-in-review, not test-first)
Illustrative only	A placeholder "library X < 2.9.1" scenario is clearly labelled as simulated; no CVE ids or scores are invented anywhere
Rollback plan	Revert one pom line / `helm rollback`; no data migration involved

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

26. Architecture mind map

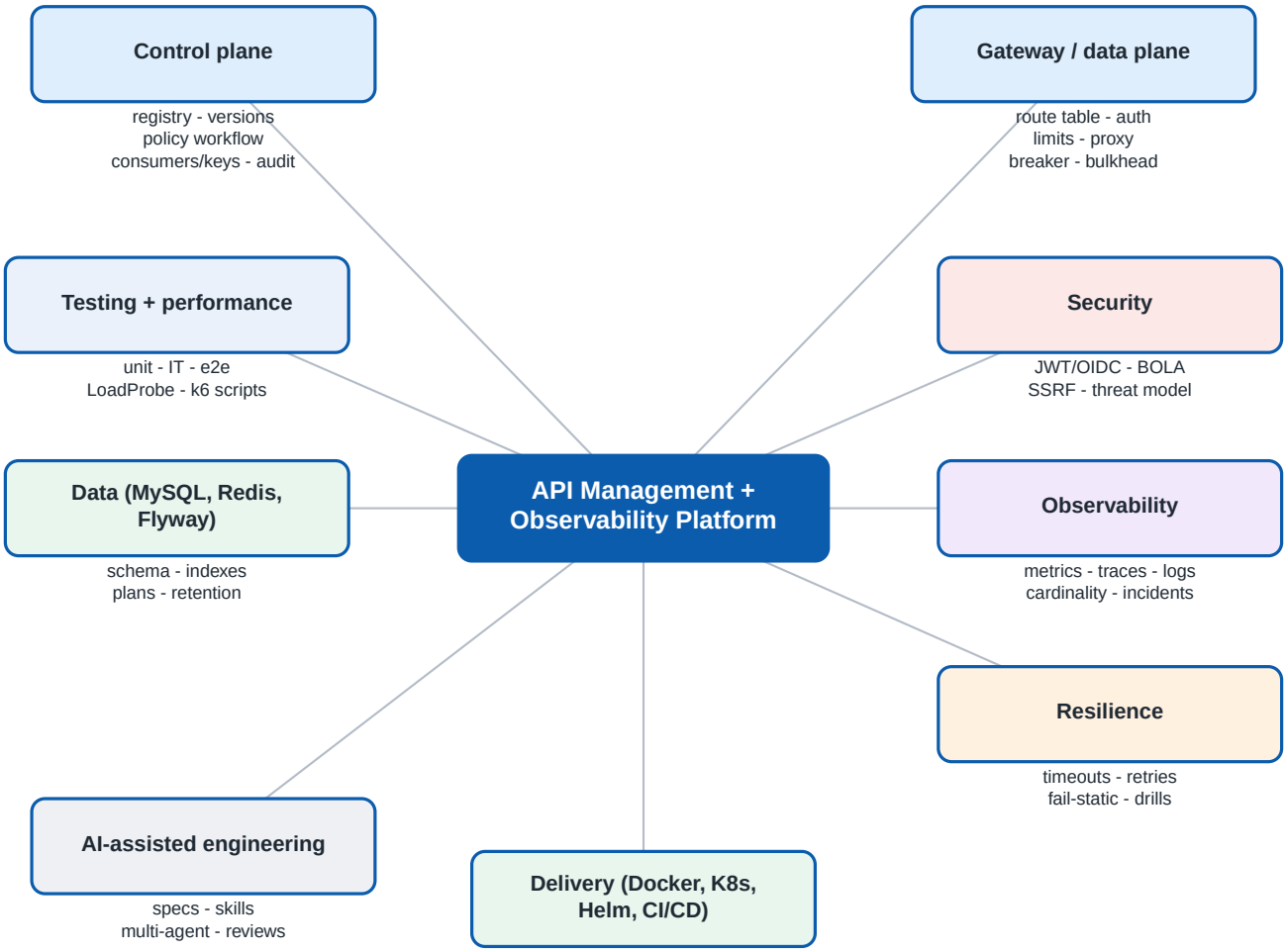


Figure 16. Mind map of the platform: control plane, gateway, security, observability, resilience, data, delivery, testing and AI-assisted engineering.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

27. Troubleshooting scenarios

#	Scenario	First evidence to collect	Typical corrective action
1	Intermittent 502/504	502 vs 504 split, TARGET_UNHEALTHY/ CIRCUIT_OPEN events, backend latency vs timeoutMs	Remove bad target, tune timeout from p99, outlier ejection
2	Latency after a release	p95 before/after, trace split gateway vs backend, audit/deploy history	Rollback; cache safe-DNS results; perf gate
3	Redis limiter inconsistent	Redis failure counter, events, X-RateLimit headers, EXPECTED_REPLICAS vs HPA	Align replica config, managed Redis, CLOSED mode where required
4	MySQL pool exhaustion	hikari pending/usage, processlist, slow log, thread dump	Paginate/index, short transactions, then right-size pool
5	High JVM memory	heap after GC, class histogram, heap dump, key counts	Bound per-key state, fix XFF trust, size limits
6	Excessive GC	GC logs/JFR allocation, humongous allocations	Pre-size/limit buffers, smaller body limits
7	Broken policy deployment	Recent config changes, policy diff, snapshot version/age	Revert (new version), show per-second effective rate in review
8	Trace correlation broken	Backend headers (/echo), propagator config, sampling	Strip + inject traceparent, W3C everywhere
9	Pods restarting	describe pod last state, previous logs, secret keys, memory vs limit	Fix config/limits; keep startupProbe
10	Upgrade regression	dependency:tree diff, release notes, old vs new on same inputs	Pin/rollback, add boundary tests, canary

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

> **Note:** All ten are exercises with synthetic root causes (`docs/troubleshooting/`), each with symptoms, hypotheses, evidence, commands, metrics/logs, root cause, corrective action, regression tests and prevention. They are not descriptions of real incidents.

28. Limitations and future enhancements

Not verified in the build environment

- Docker builds, docker compose, Kubernetes manifests, Helm chart rendering/installation, the CI workflow, Grafana/Prometheus/Jaeger runs.
- Redis Lua scripts and MySQL migrations on real servers (Testcontainers tests exist and auto-skip without Docker).
- k6 load tests (scripts only); MySQL/Redis performance; production-scale capacity.
- Live OIDC integration; penetration testing; Maven vulnerability scanning; backup/restore and failover drills.

Known design limitations

- Gateway buffers bodies (no streaming/SSE/WebSocket); HTTP/1.1 upstream only; HTTP/2/3 are conceptual.
- Consumers are not team-scoped; no developer portal; daily quotas specified but not implemented.
- Dashboard stores sum/max per minute only; percentiles come from Prometheus.
- Per-request DNS re-check costs latency on slow resolvers; consecutive-failure breaker misses interleaved partial failures.
- Gaps in tests: database-failure mid-request, telemetry shipper retry/drop logic, graceful shutdown.

Future enhancements

- OIDC + mTLS between components; signed snapshots; tail sampling at an OpenTelemetry Collector; retry budgets and outlier ejection; canary policy rollout per gateway group.
- Quota plans, consumer portal, per-team consumer scoping, WebSocket/streaming routes (possibly via a second data-plane engine).
- Pre-install migration Job, GitOps (Argo CD), policy-as-code checks in CI, chaos drills as release gates.
- Token-usage measurements for the AI context modules and measured productivity/quality comparisons.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.